

EFDI — Complete Setup Guide (New Machine)

Follow every step in order. Each step explains **what** to do and **why** it is needed.

Prerequisites at a Glance

Requirement	Minimum Version	Why
Docker Desktop	24+	Runs all three services (Postgres, backend, frontend/nginx) in containers
Docker Compose	v2 (bundled with Docker Desktop)	Orchestrates the multi-container setup from a single command
Git (optional)	Any	Only needed if you clone; not needed if you use the zip
8 GB RAM	—	EasyOCR loads a ~200 MB model into memory; Postgres + backend need headroom
4 GB free disk	—	Docker images + EasyOCR model weights + Postgres data

Windows users: Install Docker Desktop with the WSL 2 backend enabled.

Mac (Apple Silicon / ARM64): Docker Desktop supports ARM64 natively — no extra flags needed.

Linux: Install Docker Engine + the Compose plugin (`docker compose`, not `docker-compose`).

Step 1 — Extract the Project

What: Unzip the project to a working folder.

Why: The zip contains the full source tree (`efdi/`) with backend, frontend, Docker configuration, and the `.env` file all in one place.

```
bash
```

```
# Unzip (any method – Finder/Explorer drag-out works too)
```

```
unzip efdi.zip
```

```
# You should now have:
```

```
efdi/
```

```
|— backend/
```

```
|— frontend/
```

```
|— docker-compose.yml
```

```
|— .env
```

```
|— .env.example
```

```
|— README.md
```

```
|— TECHNICAL_README.md
```

```
|— FUNCTIONAL_README.md
```

Step 2 — Install Docker Desktop

What: Download and install Docker Desktop from <https://www.docker.com/products/docker-desktop>.

Why: Docker is the only runtime dependency you need on the host machine. Everything else (Python 3.12, Node 22, PostgreSQL 16, Nginx, EasyOCR, all Python and npm packages) runs **inside containers** that Docker builds from the Dockerfiles in the project. You do not install Python or Node on your laptop at all.

After installing:

- **Mac/Windows:** Launch Docker Desktop and wait for the whale icon in the menu bar to show "Docker Desktop is running".
- **Linux:**

```
bash
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
# Add yourself to the docker group so you don't need sudo every time
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

Verify:

```
bash
```

```
docker --version          # should say Docker version 24+
```

```
docker compose version    # should say Docker Compose version v2.x
```

Step 3 — Review and Configure the `.env` File

What: Open `efdi/.env` in any text editor and check the values.

Why: The `docker-compose.yml` reads this file to configure all three containers. If `POSTGRES_PASSWORD` or `SECRET_KEY` is missing or wrong, `docker compose up` will fail immediately with a clear error (the compose file uses `[:?]` syntax that enforces these are set).

The `.env` file that ships in the zip is already filled with working defaults:

```
env

# --- Database ---
POSTGRES_USER=efdi_user
POSTGRES_PASSWORD=efdi123          # Change this for any non-local deployment
POSTGRES_DB=efdi

# --- Security ---
SECRET_KEY=5af9dc0473edd08...     # Already generated – change for production

# --- Application ---
APP_ENV=production
APP_DEBUG=false
ACCESS_TOKEN_EXPIRE_MINUTES=60
REFRESH_TOKEN_EXPIRE_DAYS=7
MAX_UPLOAD_SIZE_MB=25
MAX_BULK_INTAKE_FILES=100
LOG_LEVEL=INFO

# --- OCR ---
OCR_DEFAULT_ENGINE=easyocr        # Real OCR – requires INSTALL_OCR=true below
INSTALL_OCR=true                  # Bakes EasyOCR + PaddleOCR into the backend

# --- CORS ---
CORS_ORIGINS=http://localhost

# --- Ports ---
FRONTEND_PORT=80                  # The app opens on http://localhost
```

Things you may need to change:

Variable	When to change
<code>POSTGRES_PASSWORD</code>	Any shared or non-local machine — use something strong
<code>SECRET_KEY</code>	Must regenerate for any deployment that is not purely local: <code>openssl rand -hex 32</code>
<code>FRONTEND_PORT</code>	If port 80 is already in use on your machine — change to e.g. <code>8080</code> , then access via <code>http://localhost:8080</code>
<code>OCR_DEFAULT_ENGINE</code>	Set to <code>stub</code> if you want a fast first boot with placeholder OCR output; set to <code>easyocr</code> for real recognition
<code>INSTALL_OCR</code>	Must be <code>true</code> if <code>OCR_DEFAULT_ENGINE</code> is <code>easyocr</code> or <code>paddleocr</code> ; set to <code>false</code> for a smaller/faster image with stub-only OCR

What is `stub` OCR? The backend has a stub OCR engine that returns placeholder text immediately without running any ML model. It is useful for testing the UI and workflow without waiting for a real OCR model to download. All pipeline stages (classification, extraction, validation, approval) work correctly in stub mode — you just get fake text instead of real extracted content.

Step 4 — Build and Start All Services

What: Run one command from inside the `efdi/` directory.

Why: `docker compose up --build` reads `docker-compose.yml` and `.env`, builds the backend and frontend Docker images from their respective `Dockerfile`s, pulls the PostgreSQL 16 Alpine image, and starts all three containers in dependency order (Postgres first → backend → frontend).

```
bash
cd efdi
docker compose up --build
```

What happens during the build (first run only):

Stage	What is happening	Time estimate
Postgres image pull	Downloads <code>postgres:16-alpine</code> from Docker Hub	~30 s
Backend builder stage	Installs all Python packages from <code>requirements.txt</code> into a <code>venv</code>	~3–5 min
EasyOCR + PaddleOCR install	Installs OCR packages from <code>requirements-ocr.txt</code> (if <code>INSTALL_OCR=true</code>)	~3–5 min
CPU-only torch install	Downloads <code>torch</code> CPU wheel (~190 MB) from PyTorch's index — not from PyPI, to avoid pulling in CUDA libraries	~2–4 min
Frontend builder stage	Runs <code>npm ci</code> (installs all Node packages from <code>package-lock.json</code>) then <code>npm run build</code> (TypeScript compile + Vite bundle)	~2–3 min
Nginx image	Copies the built frontend <code>dist/</code> into an Nginx Alpine image	~10 s

Why does torch install from a special URL? The default PyPI `torch` wheel links against NVIDIA CUDA libraries (`libcudart.so`, `libcublasLt.so`). On a machine without an NVIDIA GPU and drivers, `import torch` crashes at import time — even if you pass `gpu=False`. The CPU-only wheel from `https://download.pytorch.org/whl/cpu` is ~190 MB instead of several GB and imports cleanly on any machine. The Dockerfile handles this automatically.

Subsequent starts (after the first build) are fast — Docker reuses the cached image layers and only rebuilds stages where source files changed.

Step 5 — Watch the Startup Logs

What: Leave the terminal open and watch the output. You are waiting for three health checks to pass.

Why: The containers start in dependency order with health checks:

- `postgres` must be accepting connections before `backend` starts
- `backend` must return HTTP 200 on `/health` before `frontend` starts

You will see something like this when everything is ready:

```
efdi-postgres-1 | database system is ready to accept connections
efdi-backend-1  | INFO  [alembic] Running upgrade -> 4cf4fda96099 ...
(users table)
efdi-backend-1  | INFO  [alembic] Running upgrade 4cf4fda96099 ->
2ffaeae37b7d ...
efdi-backend-1  | INFO  [alembic] Running upgrade ... -> b9c2ae51cb79 ...
(training_examples)
efdi-backend-1  | INFO  [alembic] Done
efdi-backend-1  | [gunicorn] Booting worker with pid: ...
efdi-frontend-1 | Starting nginx ...
```

What is Alembic doing? The backend's Docker `CMD` runs `alembic upgrade head` before starting Gunicorn. Alembic reads all migration files in `backend/alembic/versions/` and applies them in order to create the full database schema (users, documents, ocr_results, classification_results, extraction_results, validation_results, audit_logs, workflow_history, training_examples). This runs every time the container starts but is idempotent — if the schema is already up to date, it completes in under a second.

Step 6 — Open the Application

What: Open your browser.

Why: The frontend Nginx container is bound to `FRONTEND_PORT` (default: 80) on your host machine. All API calls from the frontend go to `/api/...` which Nginx proxies internally to the backend container — your browser never talks to the backend directly.

```
http://localhost
```

If you changed `FRONTEND_PORT` to e.g. `8080`:

```
http://localhost:8080
```

You should see the EFDI login page.

Step 7 — First Login / Create a User

What: The database starts empty — no users exist. You need to create the first user.

Why: The app uses JWT-based authentication. Every API call requires a valid token, so you must register before anything else works.

Open the registration page in the UI, or use the API directly:

```
bash
```

```
# Register via API (adjust port if needed)
curl -X POST http://localhost/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "admin",
    "password": "yourpassword",
    "email": "admin@example.com",
    "full_name": "Admin User",
    "role": "admin"
  }'
```

Available roles: `admin`, `reviewer`, `submitter`

Step 8 — EasyOCR Model Download (First OCR Request Only)

What: On the very first document you upload and process with EasyOCR, there will be a delay of ~1-3 minutes.

Why: EasyOCR downloads its English recognition model weights (~100 MB) from the internet on first use. The model is saved to the `backend_model_cache` Docker volume (mapped to `/home/efdi` in the container). Because this volume persists across container restarts and rebuilds, the download only happens once for the lifetime of the deployment. Subsequent OCR calls load from the local volume and are fast.

If your machine has no internet access after setup, pre-warm the model cache before going offline:

```
bash
```

```
# Upload any test PDF after startup – this triggers the model download
# while you still have internet, and saves it to the persistent volume
```

Day-to-Day Commands

Start the application (after first setup)

```
bash
```

```
cd efdi
docker compose up -d      # -d runs in background (detached mode)
```

Stop the application

```
bash
```

```
docker compose down          # stops and removes containers; volumes (data) are
```

View logs

```
bash
```

```
docker compose logs -f      # all services
docker compose logs -f backend # backend only
docker compose logs -f frontend # nginx only
docker compose logs -f postgres # database only
```

Restart a single service (e.g. after a config change)

```
bash
```

```
docker compose restart backend
```

Check container status and health

```
bash
```

```
docker compose ps
```

Wipe everything and start fresh (destructive — deletes all data)

```
bash
```

```
docker compose down -v      # -v removes named volumes including postgres_data
docker compose up --build
```

Troubleshooting

Port 80 already in use

```
Error: bind: address already in use
```

Edit `.env`, change `FRONTEND_PORT=80` to `FRONTEND_PORT=8080` (or any free port), then:

```
bash
```

```
docker compose up -d
```

`POSTGRES_PASSWORD` error on startup

```
Error: Set POSTGRES_PASSWORD in your .env file
```

The `.env` file is missing or not in the `efdi/` directory. Make sure you are running `docker compose up` from inside the `efdi/` folder (where `docker-compose.yml` lives), not from the parent folder.

Backend container keeps restarting

```
bash
docker compose logs backend
```

Common causes:

- **Alembic migration failure** — usually a DB connection issue. Check that the `postgres` container is healthy: `docker compose ps`
 - **`SECRET_KEY` placeholder** — if you accidentally cleared the `SECRET_KEY` in `.env`, the backend startup check will reject it. Regenerate: `openssl rand -hex 32`
-

EasyOCR model download fails / times out on first OCR

The model downloads from GitHub Releases / HuggingFace on first use. If your network is slow or restricted:

- Switch to `OCR_DEFAULT_ENGINE=stub` in `.env` and `docker compose restart backend` to use placeholder OCR while you sort out connectivity.
 - Or pre-download the EasyOCR model files manually and volume-mount them — see EasyOCR docs for model file locations.
-

`docker compose` not found (Linux)

You have the old standalone `docker-compose` (v1) instead of the Compose v2 plugin. Install the plugin:

```
bash
```

```
sudo apt-get install docker-compose-plugin # Ubuntu/Debian
```

Then use `docker compose` (space, not hyphen).

502 / 504 gateway errors when uploading large files or during OCR

This is the Nginx → backend timeout issue. The `nginx.conf.template` already sets `proxy_read_timeout 180s` — if you still see 504s, the OCR is taking longer than 180 s, which can happen on very slow machines. You can increase it by editing `frontend/nginx.conf.template`, rebuilding the frontend image, and restarting:

```
bash

# After editing nginx.conf.template:
docker compose up --build frontend -d
```

Architecture Reminder

```
Browser
  | HTTP (port 80 / FRONTEND_PORT)
  ▼
frontend container (Nginx)
  | serves React app (static files from /usr/share/nginx/html)
  | proxies /api/* → backend:8000/api/* (internal Docker network)
  ▼
backend container (Gunicorn + FastAPI)
  | runs Alembic migrations on startup
  | calls EasyOCR/PaddleOCR for OCR
  | reads/writes uploads volume
  ▼
postgres container (PostgreSQL 16)
  | data persisted in postgres_data volume
```

No port on the backend or Postgres is exposed to your host machine by default. The only open port is `FRONTEND_PORT` (Nginx). This is intentional — the backend API is only reachable through the frontend's Nginx proxy, and Postgres is only reachable from the backend, both over Docker's internal bridge network.

To expose the backend API directly for debugging (e.g. to access `/api/docs` Swagger UI), uncomment the `ports` block in `docker-compose.yml` under the `backend` service:

```
yaml
```

```
# In docker-compose.yml, under backend:
```

```
ports:
```

```
- "8000:8000"
```

Then access Swagger at `http://localhost:8000/api/docs`.

*Project: Enterprise Financial Document Intelligence (EFDI) Malay | Enrollment No.
70472300159 | NMIMS MPSTME Shirpur TIP Internship — Black Box Limited, 2026*